

WebTunnel v2 Audit

Detectability of the WebTunnel pluggable transport among structurally similar legitimate traffic — practical part, rebuild review

Date 1 September 2026 **Corpus** 2,016 pilot captures · 1,546 flows **Prior audit** F-01 ... F-18

Repository /Diplomka @ 0580d28

5.5 / 10 — architecture is right, corpus is not yet admissible. Do not start Epoch A.

The v2 design is the correct design. The data contract, the gate concept, the unified Go/uTLS generator and the flow-record abstraction are all real progress, and one genuinely excellent scientific result has emerged from the pilot that was not visible in v1.

But the pilot corpus fails on the same three axes the v1 audit named, and — the more serious problem — **the gate suite currently reports PASS on four gates that cannot fail**. Gate A has not been met. The remaining work is small, roughly 250 lines across six files, but it is strictly blocking.

CONTENTS

- | | |
|------------------------|-----------------------------|
| 1 Executive summary | 2 Findings matrix F-01...18 |
| 3 New v2 findings | 4 The 100 % forensics |
| 5 Defences & base rate | 6 Pre-campaign checklist |
| 7 What must survive | |

01 Executive summary

What genuinely improved

MEASURE	V1	V2 PILOT
Positive-class captures containing a TLS handshake	0–2.4 %	56.6 %
Nominal distinct positive-class sockets	~13	275

MEASURE	V1	V2 PILOT
Cross-class contamination inside negative captures	31–52 %	0 %
Multicast / UDP noise merged into the “flow”	present	eliminated
CV group vector alignment (F-11a)	6.85 % correct	100 % correct
Transformer key-padding mask (F-16)	absent	implemented
Negatives overlapping the Tor cell band (<code>up_len_p50</code> ∈ [500, 620] B)	none	39 %

That last row is the best thing in the rebuild and it is under-celebrated: the hard-negative design finally works.

`up_len_p50` — the feature that scored AUC 1.0000 in v1 — is down to **AUC 0.8744**. The negatives now genuinely live in the Tor cell size band.

What still blocks the defence

1. **The positive class is still 8–12 independent TCP connections, not 296 flows** (F-01, unresolved). One client port, `56446`, appears in **234 of 336** WebTunnel captures and spans train, val *and* test. Herfindahl effective sample size: **7.73 sockets**.
2. **The WebTunnel client is still not using uTLS** (F-03, unresolved). Its ClientHello is still exactly **267 B** — bit-for-bit the stock Go `crypto/tls` fingerprint the v1 audit measured. G1 fails, correctly and unambiguously.
3. **The 100 % accuracy is a laboratory artefact, and it is not the 558 B invariant**. It is `up_len_max > 951`, and that threshold exists because three literal constants in `main.go` cap every negative class’s upstream TLS records at 830 B.
4. **Four of six gates pass vacuously**. G5 compares an array to a concatenation the builder itself constructed; G6 compares manifest fields to fields copied from that manifest; G2 passes only because six leaking features were added to the invariant registry; G3’s second control is non-binding and returns 1.0000.
5. **The gates are not wired into the build**. `run_full_benchmark.py` has 15 phases and `checks/run_gates.py` is in none of them. A corpus with G1 and G4 red produced three trained models and six LaTeX tables without obstruction. Principle P3 is unimplemented.
6. **The defence chapter is unchanged from v1**. `recompute_tabular_features()` — which the rebuild plan explicitly listed for deletion — is still present and still used, so before/after are still computed by two different pipelines. There is still no adaptive adversary anywhere in the repository.
7. **The host-based aggregation is still not Bayesian**. `evaluate_base_rate_fallacy.py` never loads a model; it hardcodes `llr_mean_pos = 3.5·M`, `llr_var = 4·M` and computes Gaussian tails from invented parameters.

The one result worth protecting

The pilot revealed something better than “558 B is the median”. WebTunnel’s TLS record lengths lie on an exact arithmetic **lattice**:

$L = 44 + 514 \cdot k$		44 = 5 (TLS header) + 22 (WS/HTTP framing) + 1 (inner type) + 16 (AEAD tag)						
k =	1	2	3	4	5	6	7	
	558	1072	1586	2100	2614	3128	3642	← all seven observed in the pilot

74.1 % of WebTunnel <i>upstream</i> records sit on the lattice	57.5 % of WebTunnel <i>downstream</i> records sit on the lattice	0.0–0.3 % for every legitimate class	0.9899 TPR of the zero- parameter rule at 0 / 1,250 false positives
---	--	---	---

A rule with no machine learning — “ $\geq 20\%$ of a flow’s upstream TLS records satisfy $(L-44) \bmod 514 = 0$ ” — gives TPR 0.9899, FPR 0/1,250, ROC-AUC 0.9965, with a Clopper–Pearson 95 % upper bound on FPR of 2.39×10^{-3} .

REFRAME FOR THE THESIS

This is the contribution. It is verifiable from first principles, needs two instructions to implement, is the strongest possible answer to “*your AUC is 1.0 — what single feature does that?*”, and it makes the censor-cost argument in the cascade chapter for free.

02 Audit findings matrix — F-01 ... F-18

2 fixed		4 partially addressed		12 unresolved or regressed
#	SEV	FINDING	STATUS IN V2	VERIFICATION EVIDENCE
F-01	C	Positive class is a handful of TCP connections sliced into windows	UNRESOLVED	Port 56446 in 234/336 captures; top-3 ports carry 52 % of 703 observations; Herfindahl effective $n = 7.73$; 32 sockets span >1 split ; 49.4 % of captures have no client SYN. <code>pskill -HUP tor</code> reloads config — it does not close OR connections.
F-02	C	Negatives have a TLS handshake, WebTunnel does not	PARTIAL	Improved from 0–2.4 % to 56.6 % (190/336). Asymmetry persists (negatives 99.7–100 %), and the 6-byte ChangeCipherSpec record now separates at AUC 1.0000 under a socket-disjoint split (<code>down_len_min</code> , thr 11.5).
F-03	C	WebTunnel client is not using uTLS	UNRESOLVED	G1 output: <code>webtunnel ClientHello</code> lengths: [267] — identical to v1. <code>client/torrc:8</code> carries no <code>utls</code> option. The generator's <code>dialUTLS()</code> is never on the WebTunnel path; that path goes through Tor's SOCKS proxy, so the TLS on the wire is the PT's, not the generator's.
F-04	C	No flow demultiplexing	FIXED (NEG) REGRESSED (POS)	Negatives: exact 5-tuple match, clean. Positives: <code>target_5tuple</code> hardcoded to <code>["172.20.0.3", 0, ...]</code> (wrong IP — client is <code>.30</code>), and <code>sanitizer.py:119</code> treats <code>sport == 0</code> as “match anything to bridge:443”. Result: 81.8 % of WebTunnel captures merge ≥2 distinct sockets into one “flow” (mean 2.2, max 6).
F-05	M	TSO/GSO on; the 1500 B clamp manufactures a class signal	UNRESOLVED	<code>disable_offloads()</code> swallows all errors and never verifies. Measured: 72.7 % of <code>video_streaming</code> payload packets exceed 1500 B, max 65,160 B (v1: 83.4 %, 65,160 B). Consequence: <code>len_p90</code> and <code>down_len_p90</code> are exactly 1500 for the bulk classes — and both are now <i>registered as Tor protocol invariants</i> .
F-06	M	Hard negatives do not behave as designed	PARTIAL + NEW BREAK	Genie win: 39 % of negatives now overlap the cell band. New failure: the generator caps negative upstream records at 830 B by construction (<code>400+rand(400)</code> , <code>350+rand(400)</code> , header-only GETs) while WebTunnel reaches 1072–4178 B. Zero overlap → AUC 1.0000.
F-07	M	netem egress-only, unlimited bandwidth, mislabelled	UNRESOLVED	Still <code>tc qdisc ... dev eth0 root</code> — no <code>ifb</code> , no <code>mirred</code> , no rate ceiling. <code>config.py:80</code> still prints “Gigabit Fiber, 0 % Loss, 2 ms RTT” / “30 ms RTT” / “80 ms RTT” while the script applies 20 / 45 / 90 ms. These strings go into the plots and tables.
F-08	m	Class-blocked capture order; class-dependent attrition	PARTIAL	Class is now interleaved. Profile is still blocked by wall-clock, and profile is a headline axis. Attrition is worse than v1: <code>web_assets</code> yields 101/336 = 30.1 % vs <code>websocket_chat</code> 88.4 %. 202 manifest rows record <code>no route to host</code> .

#	SEV	FINDING	STATUS IN V2	VERIFICATION EVIDENCE
F-09	C	Task separable by one constant	UNRESOLVED	v1: <code>up_len_p50</code> = 558, AUC 1.0000. v2: <code>up_len_p50</code> down to AUC 0.8744 — a real success — but <code>up_len_max</code> now scores AUC 1.0000, acc 100.00 % at threshold 951. The constant moved; it did not disappear.
F-10	C	Countermeasure result is a pipeline artefact; defences do nothing	UNRESOLVED	<code>recompute_tabular_features()</code> still at line 141, still called on both defended arms, still round-tripping IAT through <code>expm1(x·10)</code> . Still <code>max(0.0, overhead)</code> . No adaptive adversary exists. Mode 1's <code>min(1480, orig+pad)</code> actually <i>shrinks</i> WebTunnel's 2100 B and 3642 B records — truncation, not padding.
F-11	C	CV group vector misaligned with feature matrix	FIXED (A) UNRESOLVED (B)	(a) 100 % positional agreement measured. (b) 1D-CNN and Transformer still select the best epoch by <code>val_loss</code> on the evaluation fold itself and report metrics from that fold. Groups are <code>sample_id</code> , not <code>conn_id</code> — <code>conn_ids_all</code> is saved into the <code>.npz</code> and never read.
F-12	M	0x17 cutoff does not strip the TLS 1.3 handshake	UNRESOLVED	Contract documents “the client’s first record after its Finished”; <code>sanitizer.py:184</code> implements <code>if d == 1 and idx > 3</code> , ignoring content type. Worse: <code>evaluate_post_handshake.py</code> passes <code>manifest=None</code> , so the whole experiment runs with no 5-tuple demux — on v1-style contaminated data.
F-13	M	Low-FPR regime never measured	WORSE	Test split now holds 190 negatives (v1: 1,042) → resolution floor 5.3×10^{-3} . DET curve still applies <code>np.maximum(fpr, 1e-4)</code> and <code>xlim(0.01, 100)</code> ; base-rate chapter still projects 10^{-4} and 10^{-5} . No Clopper-Pearson anywhere; CV <code>ci_95</code> upper bounds exceed 1.0.
F-14	M	“Bayesian host-based aggregation” is neither	UNRESOLVED	The <code>fpr**M</code> rule is gone; what replaced it is not an LLR sum. Lines 86–93 hardcode <code>llr_mean_pos = 3.5·M</code> , <code>llr_mean_neg = -3.5·M</code> , <code>llr_var = 4·M</code> . The file never loads a model. The <code>rho_ambient</code> patch survives as <code>max(fpr_m, 0.01 · fpr_single**(0.5·M**0.3))</code> .
F-15	M	Line-rate claim omits the cost that matters	UNRESOLVED	Still inference on a pre-computed matrix; no flow-table, reassembly or feature-extraction cost. <code>n_l2_escalated = 0</code> on 239 samples — the escalation rate, the entire economic argument for the cascade, is now measured as exactly zero.
F-16	m	Modelling details	PARTIAL	✓ Transformer key-padding mask implemented. ✗ IAT still <code>loglp(Δt)/10</code> . ✗ <code>pr_auc = 1.0000000000000002</code> — a value above 1 indicates trapezoidal integration, not <code>average_precision_score</code> .
F-17	m	Documented setup path does not execute	UNRESOLVED	<code>requirements.txt</code> unchanged: <code>torch>=2.13.0</code> , <code>xgboost>=3.4.1</code> , <code>scikit-learn>=1.9.0</code> , <code>numpy>=2.5.0</code> , <code>pandas>=3.0.0</code> — none resolve. <code>CLAUDE.md</code> §7 points at <code>requirements-working.txt</code> , which does not exist.
F-18	m	Documentation asserts things the code does not do	WORSE	<code>README.md</code> is entirely v1 (“9,000 PCAPs ... 8,500 verified flows”, “native HTTP/2 framing”, 99.0 ± 0.2 %). <code>table_class_breakdown.tex</code> still labels WebTunnel “Tor over HTTP/2 WSS”. <code>CLAUDE.md</code> says keys are not in the repo; <code>ca.key</code> and <code>server.key</code> are both tracked in git. §9 describes a <code>project/</code> layout that no longer exists.

03 New findings introduced by the rebuild

V-01 C Four of six gates cannot fail

This is the most important new finding, because it is what let everything else through.

GATE	REPORTED	ASSESSMENT
G1 stack parity	FAIL	Real and correctly failing — but weaker than it looks. <code>ja4</code> is never populated by the flow builder, so the JA4 clause is <code>len(∅) ≤ 1</code> → always true. And the gate ignores classes contributing <i>no</i> ClientHello: had the positive class stayed at v1's 0 % handshake rate, G1 would have compared negatives to negatives and passed.
G2 tripwire	PASS	Passes by registry inflation. Six features were added to <code>EXPECTED_INVARIANTS</code> , including <code>len_p90</code> and <code>down_len_p90</code> — which are exactly 1500 for <code>video_streaming</code> and <code>web_assets</code> , i.e. the <code>min(pkt_len, 1500)</code> clamp. A capture artefact is registered as a Tor protocol invariant.
G3 null controls	PASS	Label shuffle 0.4580 — inside the band, but a single seed , no repetition, no CI. The second control returns 1.0000 and is explicitly excluded from <code>passed</code> . The plan's control (same-generator / different-label) is not what is implemented.
G4 budget parity	FAIL	Real and correctly failing. Worst $p = 2.2 \times 10^{-165}$.
G5 split integrity	PASS	Vacuous. <code>build_dataset.py:152</code> constructs <code>sample_ids_all</code> as literally the concatenation the gate compares against. <code>check_disjoint()</code> — the function that would test <code>conn_id</code> disjointness — exists and is never called.
G6 provenance	PASS	Tautological. <code>sanitizer.py:169–174</code> copies <code>label</code> , <code>profile</code> and <code>dest_id</code> from the manifest into the flow; the gate then compares the manifest to itself. It never checks the 5-tuple its own docstring promises, never checks capture coverage, never logs drop reasons.

TURN THIS INTO A CHAPTER

“We wrote a validation harness, then audited the harness and found four of six assertions were unfalsifiable” is a methodological contribution, and exactly the kind of self-criticism a committee rewards. `contracts.py`’s own `assert_groups_aligned()` has the same defect: it checks only `len(groups) == n_rows`, and in v1 that was already true (`8500 == 8500`) — the assertion would have passed on the bug it was written to catch.

V-02 C WebTunnel provenance is fabricated, not recorded

1_testbed/client/generator/main.go:337–347

```
ClientIP:    "172.20.0.3",    // wrong – docker-compose assigns the client 172.20.0.30
ClientPort:  0,                // "Will be matched by sanitizer via bridge target IP"
ServerIP:    "172.20.0.10",
ServerPort:  443,
```

- The rebuild plan’s founding rule — “*the collector writes ground truth; the parser never infers it*” — is violated for the only class that matters. All 495 positive manifest rows carry `client_port: 0`.
- Because the tuple is emitted unconditionally, **a failed WebTunnel session is indistinguishable from a successful one in the manifest**. All 202 recorded failures are negatives; WebTunnel shows zero. Its true failure rate is unknown.
- `sanitizer.py:142` compensates with a hardcoded IP test (`src == "172.20.0.30" or src.startswith("172.20.0.3")`) — direction inference smuggled back in, and silently wrong if any address changes.

V-03 M manifest.jsonl is append-only across runs and 32 % duplicated

`collect_scaled_dataset.py:222` truncates the global manifest only if the file does not exist. Measured: **2,971 rows for 2,016 unique `capture_id`s — 955 duplicates**. Profile counts are 1,344 / 955 / 672 where 672 each is expected, so the file mixes an aborted campaign with a rerun. `run_gates.py` keys by `capture_id`, so the last row silently wins and nothing checks whether it describes the PCAP on disk. Every row records `git_commit: 860b5d1` — the pre-v2 commit, which does not contain the collector that produced these captures.

V-04 M No TCP desegmentation, despite the docstring

`sanitizer.py` line 2 claims “TLS Record Builder ... TCP desegmentation”.
`parse_tls_records_from_payload()` operates on **one TCP segment at a time**; there is no per-flow byte-stream buffer. A record spanning segments is counted once at its declared length from the first segment, and every continuation segment falls through to `min(pkt_len, 1500)`. With TSO still enabled this is why `down_len_p90 = 1500.0` exactly for the bulk classes. The “record length” channel is a mixture of true record lengths and clamped segment lengths — and the mixture ratio is class-dependent.

V-05 M The behaviour factor is recorded but never applied

`collect_scaled_dataset.py:199` assigns `BEHAVIOURS[sid % 3]` and writes it into every manifest. The Go generator has no `--behaviour` flag and no behaviour switch. The three levels had zero causal effect on any packet. This is worse than omitting the factor: the manifest asserts a design dimension that does not exist, and `contracts.py:43` documents it as “applied to EVERY class including the positive one”.

V-06 M Only one of four declared split axes exists

`contracts.py:182` declares `unseen_connection`, `unseen_destination`, `unseen_profile`, `unseen_epoch`. The builder emits one split, keyed on `sample_id` (the trailing filename integer), not `conn_id`. `unseen_destination` is arithmetically impossible ($\text{dest_id} \in \{\text{vhost-01}, \text{bridge-01}\}$); `unseen_epoch` is impossible (all rows are epoch A). The label `"v2-Connection-Disjoint-Anti-Leakage"` describes a property the builder does not enforce — and `assert_split_disjoint()` cannot detect the violation, because `conn_id` is derived per-capture from `first_ts` and is therefore unique for all 1,546 flows by construction.

V-07 m ALPN parity is broken by design in the generator

`dialUTLS()` receives `{"h2", "http/1.1"}` for the HTTP/2 classes and `{"http/1.1"}` for the WebSocket classes. Measured ClientHello lengths differ by exactly three bytes: **506 / 538 / 570 / 602** vs **503 / 535 / 567 / 599**. Even among negatives, the ClientHello identifies the transport. This must be fixed before G1 can ever go green.

V-08 m ClientHello length has an operator-precedence bug

```
clienthello_len = (payload[3] << 8) | payload[4] + 5      # sanitizer.py:148
# Python binds + tighter than | → (payload[3] << 8) | (payload[4] + 5)
# correct for payload[4] < 251, silently corrupt above it
clienthello_len = ((payload[3] << 8) | payload[4]) + 5    # intended
```

V-09 m Reproducibility hazards

- `collect_scaled_dataset.py:91` seeds from Python's `hash()` of a string. `PYTHONHASHSEED` is randomised per process, so the “deterministic” budget draw is not reproducible across runs.
- `go.mod` does not list `github.com/gorilla/websocket`, which `main.go:16` imports. The build works only because the Dockerfile runs `go mod tidy` at image-build time — so the pinned dependency set is not pinned, and there is no `go.sum`.
- `ca.key` and `server.key` are tracked in git.
- The per-capture `.manifest.json` sidecars live under `data/raw_pcap/`, which is gitignored. The authoritative ground truth is unversioned.

04 Forensics — why every model scores 100 %

4.1 It is not the 558 B invariant

Single-feature depth-1 stumps on the v2 test split — this is `checks/tripwire.py`’s own output:

```
up_len_max      acc 100.00 %   AUC 1.0000   thr  951   ← this is the whole story
up_len_std      acc  99.58 %   AUC 0.9896   thr 332.1
down_len_p90    acc  98.74 %   AUC 0.9843   thr 1518
len_p90         acc  99.16 %   AUC 0.9792   thr 1543
up_len_mean     acc  97.49 %   AUC 0.9687   thr 555.8
up_len_p90      acc  97.49 %   AUC 0.9375   thr 816.8
up_len_p50      acc  81.17 %   AUC 0.8744   thr 557.5   ← the documented invariant
```

`up_len_p50` — the 558 B median, the feature the thesis is built around — is now the **seventh** most discriminative feature and does not even cross the tripwire limit. The classifier is not using it.

4.2 What `up_len_max > 951` actually measures

CLASS	MAX UPSTREAM RECORD	FLows WITH MAX > 951 B
<code>direct_web_browsing</code>	830 B	0.000
<code>websocket_ticker</code>	779 B	0.000
<code>websocket_chat</code>	779 B	0.000
<code>video_streaming</code>	602 B	0.000
<code>web_assets</code>	602 B	0.000
<code>webtunnel</code>	4,178 B	0.993

Zero overlap — and the negative ceiling is a *source-code constant*:

```
runDirectBrowsing: chunkSize := 400 + mrand.Intn(400) // ≤ 799 B → 830 B on the wire
runWebSocket:      msgLen     := 350 + mrand.Intn(400) // ≤ 749 B → 779 B on the wire
runVideoStreaming / runWebAssets: header-only GETs    //           → 602 B on the wire
```

WebTunnel coalesces 2, 3, 4, 6 and 7 Tor cells into single records (1072, 1586, 2100, 3128, 3642 B). **No negative class is permitted by the generator to emit an upstream record larger than 830 B.** The classifier’s perfect score therefore answers “*did the sender emit an upstream record larger than my generator’s chunk ceiling?*” — a fact about `main.go`, not about Tor. This is F-09 in a new costume, and it is currently registered in `expected_invariants.py` as “Tor cell upstream maximum payload”, which is how G2 turned green.

4.3 The three artefacts riding along

- `len_p90` / `down_len_p90` = exactly 1500.0 for `video_streaming` and `web_assets` — the `min(pkt_len, 1500)` fallback firing on TSO super-packets (87.5 % of `video_streaming` downstream “records” are exactly

1500). Both are registered as Tor invariants. They are capture artefacts.

- `down_len_min` : WebTunnel 6 B, negatives 31–58 B. Under a socket-disjoint split this scores AUC 1.0000. Six bytes is the TLS 1.3 ChangeCipherSpec record — the exact v1 F-02 artefact at a different length. It sits in `KNOWN_ARTEFACTS` , but scores 0.854 on the repo split, so G2 never names it.
- `total_bytes` / `total_pkts` / `iat_max` : G4 fails at $p \approx 10^{-165}$, so the volumetric channel remains fully class-informative. Median `total_bytes` : WebTunnel 66,871; `direct_web_browsing` 16,673; `video_streaming` 182,535.

4.4 What the honest signal is

CLASS	UPSTREAM RECORDS ON THE 44 + 514·K LATTICE	DOWNSTREAM
<code>webtunnel</code>	74.1 %	57.5 %
<code>websocket_ticker</code>	0.3 %	0.0 %
<code>direct_web_browsing</code>	0.1 %	0.0 %
<code>websocket_chat</code>	0.1 %	0.1 %
<code>video_streaming</code>	0.0 %	0.0 %
<code>web_assets</code>	0.0 %	0.0 %

rule	TPR	FPR (n = 1,250)	ROC-AUC
≥ 50 % of upstream records == 558 B exactly	0.7804	0 / 1,250	0.9895
≥ 20 % of upstream records on 44 + 514·k	0.9899	0 / 1,250	0.9965

The lattice rule dominates and degrades gracefully: TPR 0.9730 at threshold 0.3, 0.9358 at 0.6. The 22 % of positive flows the naive 558 B rule misses are exactly the ones where multi-cell coalescing dominates — which the lattice rule recovers. **This is a better result than 100 %, because it is falsifiable, derived, and reproducible by anyone with a packet capture.**

4.5 Is 100 % defensible before a committee?

In its current form, no. Three questions break it:

1. “Your best feature is `up_len_max` at threshold 951 B. What is the largest upstream record any of your negative generators can produce?” — 830 B, by three constants in `main.go` . There is no recovery from that answer.
2. “How many independent TCP connections are in your positive class?” — 8 to 12. One socket appears in 234 of 336 captures and is in train, val and test simultaneously. With 12 independent connections, the Clopper–Pearson 95 % CI on a perfect TPR is **[0.735, 1.000]**.
3. “You plot the DET curve to $FPR = 10^{-4}$. How many negatives are in your test split?” — 190. Resolution floor 5.3×10^{-3} , i.e. the plot extends 50× below the data.

AFTER THE P0 FIXES – THE ANSWER THAT WINS THE ROOM

“We do not report 100 %. We report a protocol invariant with an arithmetic derivation — record length = 44 + 514k — that holds for 74 % of WebTunnel’s upstream records and 0.1 % of legitimate traffic, measured across N independent connections, three network profiles and eight destinations, at a measured FPR below 10^{-4} with 95 % confidence. Here is the tripwire table showing what else separates and why each one is registered or was eliminated. Here is the label-shuffle control at 0.50.”

4.6 Why cross-profile (98.9 % / 96.1 %) and post-handshake (99.16 %) cannot be cited

`evaluate_cross_profile.py:56` and `evaluate_post_handshake.py:49` both call `extract_raw_packets_from_pcap()`, which invokes the flow builder with `manifest=None`. That disables 5-tuple demultiplexing entirely and falls back to the v1 behaviour — every TCP packet in the file, direction inferred from the first SYN. Both experiments run on a *different and contaminated dataset* from the headline results. The 1D-CNN’s drop to 98.9 % on LTE and 96.1 % on lossy WAN is measured on merged, non-demultiplexed host-window data and cannot be interpreted as domain generalisation.

05 Defences and base-rate aggregation

5.1 evaluate_before_after_defenses.py — F-10 unchanged

REBUILD PLAN REQUIRED	SHIPPED
Delete <code>recompute_tabular_features()</code>	Still present (line 141), still called on both defended arms
Defences applied at TLS-record level on raw traces	Applied to the normalised 200×2 tensor, post-truncation, post- <code>/1500</code>
Static and adaptive adversary side by side	No retraining anywhere; no <code>adversary</code> field; no results JSON emitted
Bandwidth and latency overhead	Bytes only; <code>max(0.0, overhead)</code> still hides byte removal

One mechanism worth naming: Mode 1's `final_bytes = min(1480.0, orig_bytes + pad)` **shrinks** WebTunnel's 2100 B and 3642 B records to 1480 B. That is not padding, it is truncation, and it is doing most of the apparent work. A defence evaluation whose “padding” reduces record sizes cannot be reported.

The v1 finding stands unmodified: against a retraining censor, 1–128 B padding moves the mode from 558 B to 559–686 B, which never enters the legitimate upstream support. The correct claim is the one the rebuild plan already wrote — **a defence must move the distribution into the legitimate support**: MTU-sized coalescing plus injected HTTP/2 control chatter to reproduce the small-record mass every legitimate class has (43 B at 30.7 % for `direct_web_browsing`, 48 B at 79.8 % for `video_streaming`) and WebTunnel entirely lacks. The pilot data supports this argument well.

5.2 evaluate_base_rate_fallacy.py — F-14 in a new form

The commit message says “implement continuous LLR aggregation”. The code does not:

```
llr_var      = m_flows * 4.0
llr_mean_pos = m_flows * 3.5
llr_mean_neg = -m_flows * 3.5
tpr_m = 1.0 - norm.cdf(0, loc=llr_mean_pos, scale=np.sqrt(llr_var))
```

Three invented constants, an unstated symmetry assumption (± 3.5), and no model is loaded by the file at all. The correlation patch survives as `max(fpr_m, 0.01 * fpr_single**(0.5 * m_flows**0.3))` — four more unmotivated constants.

What is needed is ~30 lines: take the classifier's calibrated per-flow probabilities `p_k` on the test split, group by `dest_id`, compute `S_M =  $\sum \ln(p_k / (1 - p_k))$` , sweep τ , and plot the *empirical* host-level TPR/FPR as a function of M . That needs ≥ 2 bridge destinations and ≥ 2 legitimate destinations to be meaningful — another argument for the destination-diversity work below.

06 Checklist before Epoch A / Epoch B

P0 Blocking — the campaign must not start until all are done and re-piloted ≈ 3–4 days

- 1 **Force a genuinely fresh bridge socket per WebTunnel sample.** `kill -HUP tor` reloads config; it does not close OR connections. Restart the Tor process (or the `webtunnel-client` PT), wait for bootstrap, then start `tcpdump`. *Verification gate:* every WebTunnel capture must contain a client SYN on a port not seen in any previous capture — refuse to write the PCAP otherwise. This is the single most important fix in the list.
- 2 **Record the real WebTunnel 5-tuple.** Delete the hardcoded literals in `main.go:337–347`. Snapshot `ss -tnp state established '( dport = :443 )'` inside the client container immediately after the session, or parse the PT's stdout, and write the actual client port into the manifest. Then remove `sanitizer.py`'s `t_sport == 0` wildcard branch and its hardcoded-IP direction test.
- 3 **uTLS parity for the WebTunnel client.** Verify against upstream whether the bridge line accepts a `utls` / `utls-imitate` argument; if it does, set the same Chrome profile and re-measure the ClientHello. If it does not, that is a publishable finding in its own right — report it, and fall back to the plan's contingency: strict post-handshake-only analysis with G1 relaxed to “post-handshake parity”. **Decide this before capturing anything.**
- 4 **ALPN parity across negatives.** Offer the identical ALPN list from every class, or declare HTTP/1.1-vs-h2 an explicit experimental factor and stratify on it.
- 5 **Remove the negatives' upstream record ceiling.** This is what kills the 100 %. Draw negative upstream payload sizes from a distribution that spans up to MSS — bulk POSTs, file uploads, large WebSocket frames — so `up_len_max` stops being a class label. Add an assertion that the positive and negative `up_len_max` distributions overlap.
- 6 **Actually disable offload and prove it.** `ethtool -K eth0 tso off gso off gro off` must not be error-swallowed. Add a gate: *no captured TCP payload may exceed the negotiated MSS*. Currently 72.7 % of `video_streaming` payload packets violate this.
- 7 **Implement real TCP desegmentation:** a per-direction byte-stream buffer, TLS records parsed out of the reassembled stream, and **deletion of the `min(pkt_len, 1500)` fallback**. Any segment that cannot be attributed to a record is a bug to be logged, not clamped.
- 8 **Implement real `hs_end_idx`:** track content types, find the client's `Finished`, take the first client record after it. Then *assert* that the first-*N* post-handshake record-length distributions overlap across classes. The assertion is the proof; the downstream accuracy number is not.

9 Make the gates able to fail.

- **G1** — populate `ja4` (or at minimum the ordered extension-ID list); require every class to contribute ≥ 50 ClientHellos and fail if any contributes none; fix the precedence bug at `sanitizer.py:148`.
- **G2** — prune the registry to features with a real arithmetic derivation. `up_len_max`, `up_len_std`, `len_p90` and `down_len_p90` do not have one. Replace them with a single registered lattice feature.
- **G3** — make the second control binding and change it to the plan's specification (same-generator / different-label). Run both controls over ≥ 10 seeds and report a CI.
- **G4** — switch to a *paired* budget design: draw one `(T, B_up, B_down)` triple per `sample_id` and give the identical triple to every class. The KS test then checks generator fidelity rather than the sampler.
- **G5** — call `check_disjoint()` on `conn_ids_train/val/test` (already in the `.npz`), and add the check that matters: *no client TCP port may appear in more than one split*.
- **G6** — compare the flow's *observed* 5-tuple against `manifest.target_5tuple`; assert every capture on disk produced either a flow or a logged drop reason; emit a per-class attrition table.

10 Wire `run_gates.py` into `run_full_benchmark.py` as a blocking phase immediately after `build_dataset.py`, with a non-zero exit stopping the run. This is principle P3 and it is currently absent.

11 Fix the collector's bookkeeping: truncate or rotate `manifest.jsonl` per campaign; record a structured `drop_reason`; commit the code *before* capturing so `git_commit` is meaningful; and either implement `--behaviour` in the generator or delete the field.

P1 Needed for the campaign to answer the assignment

≈ 2–3 days

1 **Destination diversity:** 8 legitimate vhosts with distinct certificates and 3 bridge instances. Without this, `unseen_destination` is impossible and the host-level LLR aggregation has nothing to group by.

2 **A QUIC / HTTP-3 negative class.** The assignment names QUIC explicitly alongside WebSockets and HTTP/2. `contracts.py` already declares `quic_http3`; the generator has no such mode. ~500 flows plus the L4-demux counterfactual and the recomputed α closes the gap in about a day.

3 **Size the campaign from the claim.** 30,000–40,000 clean negatives buys a measured “FPR < 10^{-4} at 95 % confidence”. The current 1,250 supports 2.4×10^{-3} and nothing better. Size the *positive* class by independent connections, not captures — with a fresh socket per sample those become the same number, which is the point.

- 4 **Fix attrition before scaling.** `web_assets` yields 30 %. Diagnose the `no route to host` errors on `172.20.0.20:8443` (the `legitimate-servers` container was restarting during the pilot) and add a health check before each capture.
- 5 **Interleave the profile factor**, not just the class factor, so profile is not confounded with wall-clock time and host state.

P2 Downstream — cheap once the corpus is right

≈ 3–4 days

- 1 Delete `recompute_tabular_features()` ; apply both defences at TLS-record level on raw traces; add the adaptive-adversary arm and report both side by side; measure coalescing's latency cost, not only its byte cost; remove `max(0.0, overhead)` .
- 2 Replace the invented Gaussians with $S_M = \sum \ln(p_k/(1-p_k))$ over the model's real calibrated scores, grouped by `dest_id` , swept over M .
- 3 Remove `np.maximum(fpr, 1e-4)` ; truncate the DET curve at $1/n$; report every FPR with a Clopper–Pearson interval; relabel the base-rate table as an explicitly analytical projection.
- 4 **Add the trivial lattice rule as a fourth “model”** in the generalisation matrix. An afternoon's work, and the strongest scientific anchor in the thesis.
- 5 Group `cross_validate.py` on `conn_id` ; move CNN/Transformer epoch selection to an inner fold; switch PR-AUC to `average_precision_score` .
- 6 Give `evaluate_cross_profile.py` and `evaluate_post_handshake.py` the manifest, so they use the same demultiplexed corpus as everything else.
- 7 Documentation sweep: rewrite `README.md` from v2 numbers; generate `PROFILE_DISPLAY_NAMES` from the applied `tc` parameters; fix the “Tor over HTTP/2 WSS” label; pin `requirements.txt` to versions that exist; remove the private keys from git history; update `CLAUDE.md` §7 and §9 to the current flat layout.

GATE A — GO / NO-GO, UNCHANGED

Re-run the 2,000-flow pilot after P0. **All six gates green, on gates that can fail**, and every surviving tripwire feature carrying a written arithmetic derivation. Only then start the 80-hour campaign. Expect two or three pilot iterations — that is the harness working, and it is four hours each, against a campaign that is eighty.

07 What is now genuinely good and must survive

- **The 514 B lattice.** All seven multiples observed, 74.1 % vs 0.1 % separation, TPR 0.9899 at zero false positives in 1,250 negatives with no machine learning. This is the thesis’s contribution, and it is stronger than the v1 “558 B median” framing.
- **The hard negatives finally overlap the cell band.** 39 % of negatives have `up_len_p50` $\in [500, 620]$ B, and its stump AUC fell from 1.0000 to 0.8744. That is experimental design succeeding, and it should be reported as such — it is the reason the *remaining* separation is interesting rather than trivial.
- **The unified Go/uTLS generator** is the right architecture and mostly works. Its remaining problems — the WebTunnel path, ALPN, the upstream ceiling — are three small edits, not a redesign.
- **`contracts.py` and the gate concept.** The idea is correct and ahead of much published work. The gates just need to be made falsifiable — and the story of auditing your own harness and finding four unfalsifiable assertions is a better methodology chapter than a harness that was right first time.
- **The negatives’ small-record mass** (43 B at 30.7 %, 48 B at 79.8 %) is now measured, and gives the defence chapter its concrete, testable recommendation.

Second-pass adversarial review of the WebTunnel detectability thesis, practical part · 1 September 2026 · Every figure re-derived independently of the repository’s own code: 2,016 PCAPs re-parsed with dpkt, connection identity recovered from client ports, gates executed, classifiers retrained under alternative splits and ablations. Companion to docs/01-audit-findings.md and docs/02-rebuild-plan.md.